Документация по процессору

NEON L-1

Оглавление

1. Введение ... 4
1.1 Что такое NEON L-1 ... 4
1.2 Основная идея проекта ... 4
1.3 Архитектура ... 5
1.4 Универсальные порты ... 5
1.5 Экосистема NEON ... 5
1.6 Название ... 6
2. Основные особенности ... 7
2.1 Архитектура без общей шины данных ... 7
2.2 Универсальные порты ... 7
2.3 Память ... 8
2.4 Стек вызовов ... 8
2.5 Программатор ... 8
2.6 Режимы работы ... 9
Работа через программатор ... 9
Ручной режим ... 9
2.7 Язык программирования TenBit ... 9
2.8 Программное обеспечение BitProg ... 10
2.9 Отличие от существующих решений ... 10
Ben Eater 8-bit Computer ... 10
SAP-1 computer ... 11
Gigatron TTL Computer ... 11
Magic-1 computer ... 11
Итоговое сравнение ... 12
3. Общий принцип работы ... 13
3.1 Назначение процессора ... 13
3.2 Цикл выполнения команд ... 13
Извлечение (Fetch) ... 13
Дешифрация (Decode) ... 13
Исполнение (Execute) ... 14
3.3 Организация взаимодействия модулей ... 14
3.4 Наблюдение за работой процессора ... 15
3.5 Итог ... 15
4. Визуализация работы процессора ... 16
4.1 Общий принцип ... 16
4.2 Цветовая система индикации ... 16
Белые — шины передачи данных ... 16

Оранжевые — регистры и триггеры ... 16
Красные — сигналы состояния ... 16
Зелёные — тактовый сигнал ... 17
4.3 Наблюдение за выполнением программы ... 17
4.4 Прозрачность архитектуры ... 17
4.5 Итог ... 18
5. Архитектура ... 19
5.1 Общая структура ... 19
5.2 Принцип работы ... 19
Извлечение команды ... 19
Дешифрация команды ... 19
Исполнение команды ... 20
5.3 Микрообработчики ... 20
5.4 Регистры и аккумуляторы ... 20
5.5 Итог ... 21
6. Интерфейсы и порты ... 22
6.1 Общие характеристики ... 22
6.2 Конфигурация портов ... 22
6.3 Структура порта ... 22
6.4 Протокол передачи данных в процессор (вход) ... 23
Буферизация ... 23
6.5 Протокол передачи данных из процессора (выход) ... 23
6.6 Сигнал BUSY ... 24
6.7 Синхронизация и тайминги ... 24
6.8 Параллельная работа ... 25
6.9 Распиновка порта ... 25
6.10 Ручное взаимодействие ... 25
7. Система команд (ISA) ... 27
7.1 Общая информация ... 27
7.2 Формат команд ... 27
7.3 Разделение по модулям ... 27
7.4 Выполнение команд ... 28
7.5 Таблица команд ... 28
7.6 Итог ... 29

1. Введение

1.1 Что такое NEON L-1

NEON L-1 — это учебный 8-битный процессор, созданный для наглядного изучения того, как на самом деле работает компьютер.

В отличие от обычных процессоров, которые скрывают свою работу внутри сложных микросхем, NEON L-1 делает все процессы **видимыми и понятными**.
Он позволяет наблюдать, как данные перемещаются, как выполняются команды и как процессор взаимодействует с внешними устройствами.

Процессор подходит для:

- изучения основ вычислительной техники
- понимания работы программ на низком уровне
- экспериментов с архитектурой и логикой
- создания собственных устройств и интерфейсов

NEON L-1 — это не просто микросхема, а **учебный стенд**, который показывает, что происходит внутри компьютера в реальном времени.

1.2 Основная идея проекта

Главная идея NEON L-1 — **максимальная наглядность**.

В современных системах большинство процессов скрыты. Программа выполняется, данные обрабатываются, но пользователь этого не видит.
NEON L-1 делает всё наоборот — он показывает каждый шаг.

Процессор оснащён **более чем 200 светодиодами**, которые отображают его внутреннее состояние:

- **белые** — шины передачи данных
- **оранжевые** — регистры и триггеры
- **красные** — сигналы состояния
- **зелёные** — тактовый сигнал

Благодаря этому можно буквально “увидеть”, как:

- данные проходят по шинам
- изменяются значения в регистрах
- выполняются команды
- синхронизируется работа системы

Это превращает изучение процессора из абстрактной темы в **визуальный и интуитивный процесс**.

1.3 Архитектура

NEON L-1 использует **уникальную архитектуру**, отличающуюся от классических процессоров.

Процессор разделён на **6 независимых модулей**, каждый из которых выполняет свою задачу.
Вместо одной общей шины данных используется система **отдельных соединений между модулями**.

Это даёт несколько преимуществ:

- упрощает понимание работы системы
- делает структуру более прозрачной
- позволяет легче отслеживать движение данных

1.4 Универсальные порты

Вместо классических I/O интерфейсов процессор использует **универсальные 8-битные порты**.

Эти порты представляют собой не готовые протоколы, а **базовые шины**, к которым можно подключать внешние устройства через специальные аппаратные модули (драйверы).

С их помощью можно реализовать:

- VGA (видео)
- PS/2 (клавиатура)
- UART
- I2C
- SPI
- USB (через внешний контроллер)

Таким образом, пользователь сам может “собрать” нужные интерфейсы, понимая, как они работают на низком уровне.

1.5 Экосистема NEON

NEON L-1 является частью целой экосистемы инструментов, созданных специально для него.

В неё входят:

- **программатор** — для загрузки программ в процессор
- **язык программирования «TenBit»** — разработанный специально для данной экосистемы
- **среда разработки и компилятор** — для написания и преобразования кода

Все компоненты экосистемы ориентированы на:

- простоту
- прозрачность
- обучение

В будущем планируется развитие линейки процессоров:

- **NEON L-2** — следующая версия учебного процессора
- **NEON P-1** — более углубленная и производительная версия

1.6 Название

Название **NEON L-1** происходит от слова *Learn* (обучение).

Это отражает основную цель процессора —
не максимальная производительность, а **максимальное понимание**.

2. Основные особенности

2.1 Архитектура без общей шины данных

Одной из ключевых особенностей NEON L-1 является **отсутствие общей шины данных**.

В классических процессорах все модули подключены к одной общей магистрали, по которой передаются данные. Это удобно для реализации, но усложняет понимание того, что именно происходит внутри системы.

В NEON L-1 используется другой подход:
каждый модуль соединён с другими через **отдельные линии и шины**.

Это означает, что:

- движение данных становится более наглядным
- можно точно увидеть, откуда и куда передаётся информация
- упрощается понимание архитектуры процессора

Такой подход делает процессор менее “магическим” и более **прозрачным для изучения**.

2.2 Универсальные порты

Вместо привычных интерфейсов ввода-вывода NEON L-1 использует **универсальные 8-битные порты**.

Эти порты представляют собой базовые шины, которые не привязаны к конкретному протоколу. Они предназначены для подключения внешних устройств через отдельные аппаратные модули.

Каждый порт включает:

- линии для передачи данных
- сигналы управления
- сигналы состояния

Особенность такого подхода в том, что процессор не “знает”, какое устройство подключено к порту. Он просто работает с данными, а вся логика протокола реализуется снаружи.

Это позволяет:

- подключать различные интерфейсы (VGA, PS/2, UART, I2C, SPI и др.)
- создавать собственные устройства
- глубже понимать, как работают реальные протоколы

Таким образом, порты NEON L-1 — это не просто I/O, а **универсальный механизм взаимодействия с внешним миром**.

2.3 Память

Процессор использует раздельную память для хранения программы и данных.

- **ПЗУ (EEPROM)** — 64 КБ
  Используется для хранения программы, которая выполняется процессором.
- **ОЗУ (SRAM)** — 8 КБ
  Используется для хранения данных во время выполнения программы.

Такое разделение делает структуру памяти более понятной и предсказуемой для обучения.

2.4 Стек вызовов

В NEON L-1 реализован отдельный стек, предназначенный **исключительно для вызова подпрограмм**.

Основные характеристики:

- глубина — 128 адресов
- используется только для хранения адресов возврата

Это означает, что стек не применяется для хранения данных, как в универсальных архитектурах, а выполняет строго определённую задачу.

Такой подход:

- упрощает понимание работы подпрограмм
- исключает сложные сценарии использования стека
- делает поведение процессора более предсказуемым

2.5 Программатор

Для работы с процессором используется специализированный **программатор**.

Он выполняет несколько задач:

- прошивка ПЗУ (загрузка программы)
- управление процессором с компьютера
- отслеживание внутреннего состояния
- настройка тактовой частоты

Программатор позволяет не только записывать программу, но и **взаимодействовать с процессором в реальном времени**, что особенно важно для обучения и отладки.

2.6 Режимы работы

NEON L-1 поддерживает два режима работы:

Работа через программатор

В этом режиме процессор управляется с компьютера.

Доступные возможности:

- запуск и остановка выполнения
- изменение тактовой частоты
- ручной такт
- управление сигналами
- наблюдение за состоянием системы
- наблюдение за работой процессора через светодиоды

Этот режим удобен для разработки и отладки программ.

Ручной режим

Процессор может работать полностью автономно, без подключения к компьютеру.

Управление осуществляется с помощью:

- кнопок
- тумблеров
- встроенного генератора такта

В этом режиме можно:

- тактировать процессор вручную
- наблюдать за его работой через светодиоды
- изучать выполнение программ шаг за шагом

Это превращает процессор в полноценный **физический учебный стенд**.

2.7 Язык программирования TenBit

Для программирования используется язык **TenBit**, разработанный для всей экосистемы NEON.

Особенности языка:

- простой синтаксис

- низкий уровень (близкий к архитектуре процессора)
- ориентирован на обучение

TenBit позволяет:

- понять, как код преобразуется в машинные инструкции
- работать с памятью и портами напрямую
- изучать принципы компиляции

2.8 Программное обеспечение BitProg

Для работы с процессором используется приложение **BitProg**, устанавливаемое на компьютер.

Оно объединяет несколько инструментов:

- среду разработки TenBit
- компилятор
- интерфейс взаимодействия с процессором

С помощью BitProg можно:

- писать и компилировать программы
- загружать их в процессор
- отслеживать состояние модулей
- управлять тактовым сигналом и линиями

По сути, BitProg дублирует возможности ручного управления,
но делает это **быстрее, удобнее и нагляднее**.

2.9 Отличие от существующих решений

NEON L-1 относится к классу учебных и демонстрационных процессоров, однако отличается от большинства существующих решений как по архитектуре, так и по уровню возможностей и наглядности.

Ниже приведено сравнение с наиболее известными проектами.

Ben Eater 8-bit Computer

Проекты, вдохновлённые подходом Ben Eater, ориентированы на базовое понимание принципов работы процессора.

По сравнению с ними NEON L-1:

- имеет более сложную и приближенную к реальным системам архитектуру

- поддерживает большее количество команд и возможностей
- обладает значительно более развитой системой визуализации

NEON L-1 сохраняет наглядность, но при этом выходит за рамки демонстрационной модели, приближаясь к полноценному вычислительному устройству.

SAP-1 computer

SAP-1 computer является одной из самых простых учебных архитектур и предназначен для демонстрации базовых принципов.

В сравнении с ним NEON L-1:

- имеет существенно более широкий набор команд
- поддерживает более сложные сценарии работы
- обеспечивает более детальную визуализацию внутренних процессов

Таким образом, NEON L-1 представляет следующий уровень развития учебных процессоров.

Gigatron TTL Computer

Gigatron TTL Computer — это полноценный TTL-компьютер с возможностью выполнения программ и вывода изображения.

По сравнению с ним NEON L-1:

- является более компактным и структурированным решением
- обладает более высокой степенью прозрачности (визуализация внутренних процессов)
- использует универсальные порты, которые позволяют:
    - взаимодействовать с устройствами напрямую
    - подключать аппаратные драйверы и конверторы

NEON L-1 делает акцент не только на функциональности, но и на обучении через наблюдение.

Magic-1 computer

Magic-1 computer — это более сложный самодельный компьютер с развитой архитектурой.

В сравнении с ним NEON L-1:

- является более современным учебным решением
- обладает большей наглядностью
- включает встроенную систему портов
- ориентирован не только на создание, но и на обучение

Итоговое сравнение

NEON L-1 сочетает в себе преимущества учебных и практических решений:

- более широкие возможности по сравнению с классическими учебными моделями
- высокая наглядность благодаря системе из 200+ светодиодов
- универсальные порты для взаимодействия с внешними устройствами
- собственная экосистема (программатор, язык, ПО)

В отличие от большинства аналогов, NEON L-1 может использоваться:

- как учебный стенд
- как инструмент для экспериментов
- как база для реальных проектов

Он объединяет в себе:

- функциональность
- прозрачность
- удобство разработки

NEON L-1 представляет собой не просто процессор, а **полноценную образовательную платформу**, обеспечивающую глубокое понимание работы цифровых систем.

3. Общий принцип работы

3.1 Назначение процессора

Процессор — это центральный элемент вычислительной системы, который выполняет программу.

Программа представляет собой последовательность команд, записанных в памяти.
Каждая команда — это простое действие: работа с данными, памятью или внешними устройствами.

Задача процессора заключается в том, чтобы:

- последовательно считывать команды
- интерпретировать их
- выполнять соответствующие действия

Таким образом, любое поведение системы — от простых вычислений до вывода изображения — определяется программой, а процессор выступает как исполнитель этой программы.

3.2 Цикл выполнения команд

Работа процессора организована в виде повторяющегося цикла, состоящего из трёх этапов:

извлечение → дешифрация → исполнение
(fetch → decode → execute)

Этот цикл непрерывно повторяется при выполнении программы.

Извлечение (Fetch)

На первом этапе процессор считывает очередной байт из ПЗУ.

Этот байт является частью программы и представляет саму команду.

Фактически, процессор последовательно “проходит” по программе, считывая её байт за байтом.

Дешифрация (Decode)

Считанный байт передаётся в модуль **Command Extractor (CE)**.

Задача CE:

- определить тип команды
- выбрать модуль, который должен её обработать

На этом этапе происходит **первичное распознавание команды**.
CE не выполняет её, а только маршрутизирует.

Исполнение (Execute)

После определения типа команды она передаётся в соответствующий модуль.

Внутри модуля происходит:

- окончательная дешифрация команды
- выполнение необходимого действия

Если команда требует дополнительных данных (аргументов), модуль инициирует запрос к CE.

В этом случае:

- CE извлекает следующий байт из ПЗУ
- передаёт его напрямую модулю
- при этом байт **не проходит стадию дешифрации в CE**

Таким образом:

- первый байт определяет *что делать*
- последующие байты — *с какими данными работать*

3.3 Организация взаимодействия модулей

В NEON L-1 выполнение команды реализуется через взаимодействие отдельных модулей.

Общий процесс выглядит следующим образом:

1. Байт извлекается из ПЗУ
2. Передаётся в CE
3. CE определяет тип команды
4. Команда направляется в соответствующий модуль
5. Модуль выполняет операцию
6. При необходимости запрашивает дополнительные байты

После завершения выполнения процесс возвращается к следующему байту программы.

Такой подход обеспечивает:

- чёткое разделение ответственности между модулями
- предсказуемость выполнения
- прозрачность архитектуры

3.4 Наблюдение за работой процессора

В NEON L-1 все этапы выполнения команд можно наблюдать напрямую.

Система индикации на основе светодиодов отображает:

- передачу данных между модулями
- активность регистров
- состояние управляющих линий
- тактовый сигнал
- активный модуль

Это позволяет:

- отследить извлечение команды из памяти
- увидеть её передачу в CE
- наблюдать маршрутизацию команды
- зафиксировать момент исполнения

При пошаговом тактировании (ручной режим) выполнение программы становится полностью контролируемым и визуально понятным.

3.5 Итог

Процессор NEON L-1 реализует классический цикл выполнения команд:

извлечение → дешифрация → исполнение

Однако, в отличие от традиционных решений,
данный процесс:

- явно разделён между модулями
- полностью прозрачен
- доступен для визуального наблюдения

Это делает NEON L-1 не только вычислительным устройством,
но и эффективным инструментом для изучения принципов работы процессоров.

4. Визуализация работы процессора

4.1 Общий принцип

Одной из ключевых особенностей NEON L-1 является развитая система визуализации.

Процессор оснащён **более чем 200 светодиодами**, которые отображают его внутреннее состояние в реальном времени.
В отличие от традиционных систем, где работа процессора скрыта, NEON L-1 делает все внутренние процессы **наблюдаемыми на аппаратном уровне**.

Каждый важный элемент системы — от передачи данных до изменения состояния регистров — имеет визуальное представление.

4.2 Цветовая система индикации

Для упрощения восприятия светодиоды разделены по функциональным группам с помощью цвета.

Белые — шины передачи данных

Отображают перемещение данных между модулями процессора.
Позволяют увидеть:

- откуда поступают данные
- куда они направляются
- в какой момент происходит передача

Оранжевые — регистры и триггеры

Показывают текущее состояние внутренних элементов хранения.

С их помощью можно наблюдать:

- значения регистров
- изменение битов при выполнении операций
- состояние отдельных логических элементов

Красные — сигналы состояния

Отображают управляющие и служебные сигналы.

Это могут быть:

- сигналы активности модулей

- управляющие линии
- внутренние состояния процессора

Зелёные — тактовый сигнал

Показывают работу тактового генератора.

Позволяют:

- визуально отслеживать частоту работы
- синхронизировать наблюдение за системой
- видеть отдельные такты при пошаговом выполнении

4.3 Наблюдение за выполнением программы

Система визуализации позволяет отслеживать выполнение программы на уровне отдельных шагов.

Во время работы процессора можно наблюдать:

- извлечение команды из памяти
- передачу данных в модуль дешифрации
- активацию соответствующего модуля
- выполнение операции
- изменение состояния регистров

При ручном тактировании каждый шаг выполнения соответствует одному изменению состояния системы, что позволяет детально анализировать работу программы.

4.4 Прозрачность архитектуры

Использование светодиодов превращает процессор в полностью прозрачную систему.

Пользователь может:

- проследить путь данных через все модули
- определить, какие элементы активны в данный момент
- понять причину изменения состояния системы
- выявлять ошибки на уровне аппаратной логики

Таким образом, NEON L-1 позволяет не только выполнять программы,
но и **изучать их выполнение непосредственно на уровне аппаратуры**.

4.5 Итог

Система визуализации NEON L-1 обеспечивает:

- наглядное представление всех внутренних процессов
- прямое соответствие между логикой и её отображением
- возможность пошагового анализа работы процессора

Это делает процессор не только вычислительным устройством,
но и эффективным инструментом для глубокого понимания цифровых систем.

5. Архитектура

5.1 Общая структура

Архитектура NEON L-1 построена по модульному принципу.

Процессор состоит из **6 независимых модулей**, каждый из которых реализован на отдельной плате и выполняет строго определённую функцию:

- **PC (Program Counter)** — программный счётчик
- **CE (Command Extractor)** — экстрактор команд
- **MM (Memory Manager)** — менеджер памяти
- **PM (Port Manager)** — менеджер портов
- **ALU (Arithmetic Logic Unit)** — арифметико-логическое устройство
- **AM (Address Manager)** — менеджер адресов

Модули соединены между собой через выделенные линии, без использования общей шины данных. Это обеспечивает прозрачность архитектуры и упрощает отслеживание потоков данных.

Подробное описание каждого модуля приведено в отдельной документации по архитектуре NEON L-1.

5.2 Принцип работы

Работа процессора основана на классическом цикле выполнения команд:

извлечение → дешифрация → исполнение
(fetch → decode → execute)

Извлечение команды

1. Модуль **CE** получает текущий адрес из **PC**
2. По этому адресу выполняется обращение к ПЗУ
3. ПЗУ возвращает байт команды

Дешифрация команды

Полученный байт разделяется на две части:

- **старший полубайт (4 бита)** — определяет модуль-исполнитель
- **младший полубайт (4 бита)** — определяет конкретную команду внутри модуля

На первом этапе:

- **CE дешифрирует старший полубайт**
- определяет, в какой модуль направить команду

Команда может быть передана в один из модулей:

- **PM** — работа с портами
- **MM** — работа с памятью
- **ALU** — арифметика и логика
- **AM** — работа с адресами

Исполнение команды

Модуль, получивший команду:

1. Дешифрирует **младший полубайт**
2. Определяет конкретную операцию
3. Запускает соответствующий **микрообработчик**

5.3 Микрообработчики

Каждый модуль содержит набор микрообработчиков, реализующих выполнение команд.

Особенности:

- один микрообработчик может обслуживать несколько команд
- выполнение разбито на последовательность тактов
- каждый такт управляет отдельными линиями и регистрами

Микрообработчик:

- управляет внутренними сигналами
- координирует обмен данными
- завершает выполнение команды

5.4 Регистры и аккумуляторы

В процессоре используются два основных аккумулятора:

- **A** — основной рабочий регистр
- **B** — вспомогательный регистр

Большинство операций выполняется с использованием регистра **A**.

Регистр **B** применяется в операциях, требующих второго операнда, например:

- сложение: A = A + B
- вычитание: A = A - B

Такое разделение упрощает архитектуру и делает поведение арифметических операций более предсказуемым.

5.5 Итог

Архитектура NEON L-1 характеризуется:

- модульной организацией
- отсутствием общей шины данных
- двухуровневой системой дешифрации команд
- использованием микрообработчиков для исполнения
- разделением функций между специализированными модулями

Это обеспечивает:

- понятную структуру
- прозрачность выполнения
- удобство для изучения и анализ

6. Интерфейсы и порты

6.1 Общие характеристики

NEON L-1 использует универсальные двусторонние 8-битные порты для взаимодействия с внешними устройствами.

Основные параметры:

- логический уровень: **5V**
- логика сигналов:
  - 0 — логический ноль (False)
  - 1 — логическая единица (True)
- инвертированные линии отсутствуют

Разделение управления линиями:

- **выходные линии** управляются только процессором
- **входные линии** управляются только внешним устройством

Для входных линий на стороне процессора реализована подтяжка к земле через резисторы **10 кОм**, что обеспечивает стабильное состояние при отсутствии сигнала.

6.2 Конфигурация портов

Базовая конфигурация процессора включает **3 универсальных двусторонних порта.**

Каждый порт представляет собой независимый интерфейс и может использоваться параллельно с другими.

6.3 Структура порта

Каждый порт имеет разрядность 8 бит и включает следующие линии:

- **DATA_IN[7:0]** — шина приёма данных (вход)
- **DATA_OUT[7:0]** — шина передачи данных (выход)
- **DATA_IN_SET** — сигнал фиксации входных данных
- **DATA_OUT_SET** — сигнал фиксации выходных данных
- **BUSY** — сигнал занятости (выход)
- **CLK** — тактовый сигнал (выход)
- **+5V / GND** — питание

Порт не реализует конкретный протокол и выступает как **базовый параллельный интерфейс**, на основе которого могут строиться более сложные системы.

6.4 Протокол передачи данных в процессор (вход)

Передача данных во входную шину выполняется следующим образом:

1. Проверяется состояние сигнала **BUSY**
   - если BUSY = 0 — порт готов к приёму
   - если BUSY = 1 — передача запрещена
2. Устанавливается байт на линиях **DATA_IN[7:0]**
   - допускаются переходные состояния (глитчи), так как шина в этот момент не считывается
3. После стабилизации данных формируется импульс на линии **DATA_IN_SET**
   - на **восходящем фронте** происходит фиксация данных во внутренний регистр

После фиксации:

- процессор записывает байт во внутренний буфер
- сигнал **BUSY** устанавливается в 1

Буферизация

Процессор имеет **1-байтовый входной буфер**.

Поведение:

- после приёма данных → BUSY = 1
- после чтения буфера процессором → BUSY = 0

Если BUSY = 1, новые данные передавать нельзя.

Внимание:

После чтения входного буфера командой **D2** данные в буфере **не очищаются автоматически** и могут быть прочитаны повторно.

При этом:

- сигнал **BUSY** устанавливается в 0
- команды **C5–C7** (условные переходы) **не инициируют переход**

6.5 Протокол передачи данных из процессора (выход)

Передача данных из процессора осуществляется автоматически:

1. Процессор устанавливает байт на линиях **DATA_OUT[7:0]**
2. Формируется импульс на линии **DATA_OUT_SET**
3. На **восходящем фронте** внешнее устройство должно зафиксировать данные

Особенности:

- возможны переходные состояния при установке DATA_OUT
- также возможны изменения при спаде DATA_OUT_SET
- фиксацию следует выполнять именно по фронту сигнала

Передача полностью синхронизирована с внутренними тактами процессора.

6.6 Сигнал BUSY

Сигнал **BUSY** является выходом процессора и указывает на его состояние:

- BUSY = 0 — готов к приёму данных
- BUSY = 1 — занят

Причины установки BUSY:

- обработка данных
- заполненный входной буфер

6.7 Синхронизация и тайминги

Для повышения стабильности рекомендуется синхронизировать обмен с сигналом **CLK**, однако это не является обязательным.

Основные требования:

- не выполнять запись при BUSY = 1
- обеспечивать стабильность данных перед фиксацией

Минимальные временные параметры:

- минимальное время установки/удержания сигналов — порядка **30 нс**

Примеры:

- удержание DATA_IN перед DATA_IN_SET
- длительность импульса DATA_IN_SET

Следует учитывать:

- паразитные ёмкости соединений
- длину шлейфов

Рекомендуемая рабочая частота:

- до **~500 кГц**

6.8 Параллельная работа

Порты поддерживают **одновременный двусторонний обмен**:

- процессор может передавать данные, пока принимает другие
- внешнее устройство может писать и читать независимо

Это позволяет реализовывать более сложные схемы взаимодействия.

6.9 Распиновка порта

Каждый порт реализован через разъём:

- **IDC-M 2×12, шаг 2.54 мм**

Назначение выводов:

Пин	Назначение
1–8	DATA_IN[0..7] — вход
9	GND
10	DATA_IN_SET — вход
11–18	DATA_OUT[0..7] — выход
19	GND
20	DATA_OUT_SET — выход
21	+5V
22	BUSY — выход
23	GND
24	CLK — выход

6.10 Ручное взаимодействие

Благодаря простой структуре интерфейса, взаимодействие с портами возможно не только через внешние устройства, но и напрямую.

Пользователь может:

- подавать сигналы вручную
- наблюдать линии через светодиоды
- тестировать обмен без дополнительной электроники

Это делает порты удобным инструментом для:

- обучения

- отладки
- экспериментов

7. Система команд (ISA)

7.1 Общая информация

Для процессора NEON L-1 разработана собственная система команд (Instruction Set Architecture, ISA).

ISA определяет набор инструкций, которые процессор может выполнять на аппаратном уровне. Именно из этих инструкций состоит любая программа, выполняемая процессором.

Каждая команда представляет собой один или несколько байтов, которые последовательно считываются из ПЗУ и исполняются процессором.

7.2 Формат команд

Команды в NEON L-1 могут иметь переменную длину:

- **1 байт** — простые команды без аргументов
- **2 байта** — команда + 1 аргумент
- **3 байта** — команда + 2 аргумента

Первый байт всегда содержит:

- тип команды
- указание на модуль-исполнитель

Дополнительные байты (при наличии) используются как данные для выполнения команды.

7.3 Разделение по модулям

Система команд NEON L-1 построена в соответствии с архитектурой процессора.

Все команды логически разделены по модулям:

- **PM (Port Manager)** — команды работы с портами
- **MM (Memory Manager)** — команды работы с памятью
- **ALU (Arithmetic Logic Unit)** — арифметические и логические операции
- **AM (Address Manager)** — команды работы с адресами

Определение модуля выполняется на этапе дешифрации:

- **старший полубайт команды** указывает, в какой модуль она направляется
- **младший полубайт** определяет конкретную операцию внутри модуля

Такой подход обеспечивает:

- логическую структурированность команд
- упрощённую дешифрацию
- соответствие архитектуре процессора

7.4 Выполнение команд

После определения модуля команда передаётся соответствующему блоку, где:

1. Выполняется окончательная дешифрация
2. При необходимости запрашиваются дополнительные байты
3. Запускается микрообработчик
4. Команда исполняется по тактам

7.5 Таблица команд

Полный список команд приведён в таблице ниже.

Команда	HEX	Binary	Описание
LDA@	A1	10100001	Загрузка аккумулятора A в ОЗУ по адресу
LDB@	A2	10100010	Загрузка аккумулятора B в ОЗУ по адресу
EXA@	A3	10100011	Выгрузка из ОЗУ по адресу в аккумулятор A
EXB@	A4	10100100	Выгрузка из ОЗУ по адресу в аккумулятор B
SETA&	A5	10100101	Загрузка значения в аккумулятор A
SETB&	A6	10100110	Загрузка значения в аккумулятор B
ADD	B1	10110001	Прибавить число B к аккумулятору A (A=A+B)
SUB	B2	10110010	Отнять число B от аккумулятора A (A=A-B)
INV	B3	10110011	Инвертировать аккумулятор A
SET0$	B4	10110100	Установить биты в 0 в аккумуляторе A (AND)
SET1$	B5	10110101	Установить биты в 1 в аккумуляторе A (OR)
SHL	B6	10110110	Сдвинуть биты в аккумуляторе A на 1 значение в лево
SHR	B7	10110111	Сдвинуть биты в аккумуляторе A на 1 значение в право
JMP@	C1	11000001	Безусловный переход на адрес
IF=JMP@	C2	11000010	Переход на адрес, если A = B
IF-JMP@	C3	11000011	Переход на адрес, если A<B
IF+JMP@	C4	11000100	Переход на адрес, если A>B
IF1JMP@	C5	11000101	Переход на адрес, если есть данные на 1 порту
IF2JMP@	C6	11000110	Переход на адрес, если есть данные на 2 порту
IF3JMP@	C7	11000111	Переход на адрес, если есть данные на 3 порту
STR@	C8	11001000	Запуск подпрограммы по адресу
EXT	C9	11001001	Выход из подпрограммы
OUT&	D1	11010001	Отправить аккумулятор A в порт
IN&	D2	11010010	Прочитать порт и записать в аккумулятор A
NOP	FF	11111111	Задержка
STOP	00	00000000	Остановка процессора до сброса

7.6 Итог

Система команд NEON L-1:

- разработана специально для данной архитектуры
- поддерживает переменную длину инструкций
- разделена по функциональным модулям
- напрямую связана с внутренней структурой процессора

Это делает ISA:

- понятной
- расширяемой
- удобной для изучения